WEEK 2 · DAY 2

LLM Pipelines & AI FDE Workflows

From a Business Problem to a Working Copilot

1-Hour Lecture Session — Concepts & Workflow (No Live Platform Demo)

Rajesh Pasham

Program Context

How today's session fits the wider 8-week delivery

Session Type	Concepts and workflow lecture — no live Palantir platform work today
Session Format	One combined live session for India & Europe
Platform	Individual Palantir Developer Tier accounts, used from Day 3 onward
This Week's Lab (Day 3)	Build your batch's Copilot for real — today's workflow is exactly what you'll follow

Where Today Fits

Day 1 covered architecture. Today covers the workflow that builds it.

Day 1 gave you the architecture vocabulary — tokens, prompt layers, grounding, safety. Today gives you the process: how a real operational requirement actually becomes a working Copilot, step by step, and what "done" looks like before you escalate to a human instead of guessing.

Today's Roadmap

Four topic groups, ending exactly where Day 3 begins

1	**LLM Pipeline Design**	Turning ad hoc prompting into a reusable, governed pipeline
2	**The AI FDE Workflow**	How operational requirements become AIP use cases
3	**From Prototype to Production**	Rapid prototyping, testing with real business users, and what changes after
4	**Building the Copilot**	What “functional” means, and how a Copilot should fail gracefully

By the End of Today

The workflow you'll actually follow in Day 3, not just discuss

Design a reusable LLM pipeline

Not a one-off prompt — a pattern your batch can reapply across every question your Copilot needs to answer

Translate an operational requirement into an AIP use case

The specific move from “adjusters spend too long on X” to a scoped, buildable Copilot capability

Explain why prototypes get tested with real business users

Not just other engineers — and what changes between a prototype and an operational solution

Design a Copilot's limitations and escalation path deliberately

Before Day 3, not as a fix after something goes wrong

TOPIC 1

LLM Pipeline Design

Turning ad hoc prompting into a reusable, governed pipeline

Designing Reusable LLM Pipelines

A pipeline, not a single clever prompt

One prompt answers one question well; a pipeline answers a family of questions

Your batch's Copilot needs to handle variation — different claims, applications, or cases — not just the one example you tested

Each stage should be independently testable

Data preparation, retrieval, and prompt assembly are separate stages — debug them separately, not as one opaque block

Reusable means versioned and shared

The same pipeline pattern from Day 1's prompt template applies here — fixed structure, varying content

The Pipeline, Stage by Stage

Where today's four stages come from, and what each one does

Data Preparation	Getting Ontology objects and documents into a form an LLM can consume — covered next
Context Pipeline	The retrieval step that decides what's relevant to this specific question
Prompt Assembly	Combining system, user, and contextual instructions — Day 1's three layers, now automated

Data Preparation for LLM Consumption

Ontology objects and documents don't arrive LLM-ready

Structured Data	An Ontology object has many properties — decide which ones actually belong in the prompt, not all of them by default
Unstructured Data	Documents need the chunking and embedding pipeline from Week 4 before an LLM can retrieve from them meaningfully
The Common Mistake	Passing an entire object or document verbatim “to be safe” — this is the same context-window cost problem from Day 1, just showing up earlier in the pipeline

Prompt & Context Pipelines

Making retrieval and assembly repeatable, not manual

The context pipeline decides what's relevant, per call

Given a claimId or applicationId, what should actually be retrieved — this logic runs every time, not once

The prompt pipeline assembles the final compiled prompt

System instructions (fixed) plus retrieved context (variable) plus the user's question — automated, not hand-typed per call

Ontology-Aware Retrieval

Retrieval that understands relationships, not just keywords

Retrieval can follow Link Types, not just match text

Given a Claim, ontology-aware retrieval can pull its Policy and Policyholder automatically — the relationships from Week 1 become retrieval logic

Governance-aware by construction

If a property is Marking-protected, ontology-aware retrieval respects that — it isn't a separate check bolted on afterward

Function & Tool Integration

Where a pipeline stops retrieving and starts acting

A tool is a capability the model can invoke, not just data it reads

A lookup Function, a calculation, or (from Week 3 onward) an Action — the model decides when to call it, based on the tools you've exposed

Describe each tool clearly enough that the model knows when to use it

A vague tool description leads to a model either ignoring a useful tool or calling the wrong one

TOPIC 2

The AI FDE Workflow

How operational requirements become AIP use cases

The AI Forward Deployed Engineering Workflow

A discipline for going from a vague ask to a scoped build

This is the workflow every batch actually runs — for real — starting Day 3. Today walks each stage; Day 3 is where you do it.

Translating Operational Requirements into AIP Use Cases

The step most teams rush, and shouldn't

The Requirement, As Given	“Adjusters spend too long reviewing claim documents” — real, but not yet buildable
The Translation	What specific question, asked of what specific Ontology objects, would actually save that time? Name it precisely.
The Output	A scoped Copilot capability: “Given a claimId, summarize the claim and flag missing information” — buildable, testable, and exactly what Week 2's Practical Lab specifies

TOPIC 3

From Prototype to Production

Rapid prototyping, testing with real business users, and what changes after

Rapid Prototyping

Building the smallest thing that tests the real question

Prototype the riskiest assumption first

If you're unsure whether grounding will actually work for this use case, test that before polishing the prompt wording

A prototype is disposable by design

Expect to rebuild parts of it once you see real usage — that's the point of prototyping quickly, not a sign it failed

Testing with Business Users

Why an engineer's judgment isn't enough on its own

Domain experts catch what engineers can't

An adjuster or underwriter will notice a wrong risk judgment that reads as perfectly plausible to someone outside the domain

Watch them use it, don't just ask if it's good

What someone actually tries to ask the Copilot is more revealing than their opinion of a demo you walked them through

Real usage surfaces the edge cases your test data didn't

This is where you'll find the ambiguous questions Day 1's “handling ambiguity” topic was preparing you for

Moving from Prototype to Operational Solution

What actually has to change, not just get polished

Prototype	Answers the question, for the cases you tested, with you watching
Operational Solution	Answers the question reliably, for cases you didn't test, with governance, Evals, and observability in place — Weeks 3, 4, and 6
The Gap Between Them	Everything this program builds from Week 3 onward — writeback, document intelligence, multi-agent design, production readiness — exists to close exactly this gap

TOPIC 4

Building the Copilot

What “functional” means, and how a Copilot should fail gracefully

Building Initial Functional Copilots

What “functional” actually requires — this week's Practical Lab spec

Accepts an identifier and retrieves grounded data

A claimId, applicationId, fraudCaseId, or policyholderId — the entry point for your batch's Copilot

Summarizes, flags, and recommends — doesn't decide

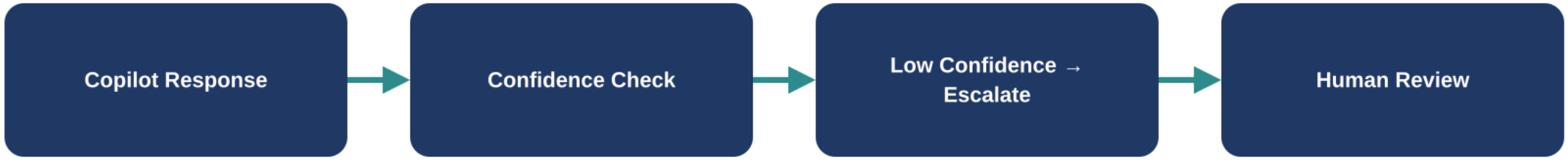
This week's Copilot reads and reasons; Week 3 is where it starts writing back

Refuses unauthorized queries

If the requester's Role or Marking eligibility doesn't allow it, the Copilot says so — it doesn't quietly comply

Copilot Limitations & Escalation Paths

Designing where the Copilot stops, before Day 3



Escalation isn't a failure state to minimize — it's a designed outcome. A Copilot that escalates a genuinely low-confidence case is working correctly; one that guesses confidently instead is the actual failure.

Common Pitfalls

Patterns worth avoiding before Day 3's build

Skipping straight to prompt-writing	Without translating the requirement first, you're guessing at scope — revisit Topic 2 before touching a prompt
Testing only with engineers	A Copilot that looks right to you may still be wrong to the domain expert who'll actually use it
No escalation path until something breaks	Design the low-confidence route now — retrofitting it after a bad answer reaches a user is too late
Treating the prototype as the deliverable	A working demo and an operational solution are different bars — know which one you're building today

Recap & What's Next

What We Covered Today

- LLM pipeline design: data prep, context, prompt assembly
- Ontology-aware retrieval and function/tool integration
- The AI FDE workflow: requirement → use case → prototype
- Testing with business users, prototype vs. operational
- Building a functional Copilot with a real escalation path

Day 3 — Self-Paced Lab

- Build your batch's Insurance Claims Copilot for real
- Follow today's exact workflow: requirement → use case → build
- Test it against your batch's own Ontology from Week 1
- Design the escalation path you learned about today

Questions

Rajesh Pasham